

Análisis de Patrones de Diseño y Arquetipos Maven

SmartLogix — DSY1106 Desarrollo Fullstack III

Evaluación Parcial N°2

1. Patrones de Diseño Aplicados

1.1 Factory Method — `OrdenFactory`

Ubicación: `Orden/src/main/java/Orden/example/Orden/factory/OrdenFactory.java`

Descripción:

La clase `OrdenFactory` centraliza la creación de los objetos del dominio (`OrdenModel`, `DetalleOrdenModel`, `HistorialModel`), evitando que el servicio instancie modelos directamente y garantizando que todos los objetos se creen con los campos obligatorios correctamente inicializados.

Métodos de fábrica:

```
OrdenFactory.crearOrden(userId, userNombre, direccionId)
OrdenFactory.crearDetalle(orden, productoId, nombre, precio, cantidad)
OrdenFactory.crearHistorial(orden, estadoId, estadoNombre, comentario)
```

Beneficio: Si la lógica de creación cambia (ej. agregar campos de auditoría), solo se modifica la factory, sin tocar el servicio.

1.2 Circuit Breaker — `Resilience4j`

Ubicación: `Orden/src/main/java/Orden/example/Orden/client/UsersClient.java` (y `EstadoClient`, `ProductoClient`)

Descripción:

Cada cliente REST que se comunica con otro microservicio usa `CircuitBreakerFactory` de Spring Cloud. Si el servicio destino falla o demora, el circuit breaker activa el **fallback**, devolviendo `null` en lugar de propagar una excepción que rompería la cadena de llamadas.

```
circuitBreakerFactory.create("usersClient").run(
    () -> restTemplate.getForObject(usersUrl + "/api/users/" + userId, Map.class),
    throwable -> { log.error(...); return null; }
);
```

Beneficio: El microservicio `Orden` puede crear órdenes incluso si `Users` está temporalmente caído, usando el nombre del request como fallback.

1.3 Repository Pattern — Spring Data JPA

Ubicación: `*/repository/OrdenRepository.java`, `HistorialRepository.java`, `DetalleOrdenRepository.java`

Descripción:

Todos los accesos a la base de datos se realizan a través de interfaces que extienden `JpaRepository`. La capa de servicio nunca escribe SQL directamente; delega en el repositorio.

```
public interface OrdenRepository extends JpaRepository<OrdenModel, Long> {
    List<OrdenModel> findById(UUID userId);
}
```

Beneficio: Desacopla la lógica de negocio del motor de persistencia. Cambiar de PostgreSQL a otro motor solo requiere cambiar el driver, no el servicio.

1.4 BFF (Backend for Frontend) — Gateway

Ubicación: `Gateway/src/main/resources/application.yml`

Descripción:

El `Gateway` actúa como punto de entrada único para el frontend Angular. Valida el JWT en el filtro `JwtGatewayFilter`, extrae `userId` y `rol`, y los inyecta como headers antes de enrutar la solicitud al microservicio correspondiente.

```
Angular (4200) → Gateway (8080) → Microservicio destino (808x)
```

Beneficio: El frontend no conoce las URLs internas de los microservicios. La autenticación se resuelve en un solo punto, simplificando la seguridad de cada servicio.

1.5 Observer — RxJS BehaviorSubject (Frontend)

Ubicación: `SmartLogix/src/app/features/ordenes/orden.service.ts`

Descripción:

`OrdenService` mantiene un `BehaviorSubject<Orden[]>` que emite la lista actualizada de órdenes cada vez que se recarga del backend. Todos los componentes suscritos (`OrdenesComponent`, `EnviosComponent`) reciben los cambios automáticamente sin necesidad de comunicación directa entre componentes.

Beneficio: Desacopla la fuente de datos de los componentes que la consumen. Al agregar un historial o tomar una ruta, el estado se propaga a todos los componentes suscritos de forma reactiva.

1.6 Interceptor HTTP (Frontend)

Ubicación: `SmartLogix/src/app/core/interceptors/`

Descripción:

`JwtInterceptor` intercepta cada petición HTTP saliente y agrega automáticamente el header `Authorization: Bearer <token>` sin que cada componente o servicio deba hacerlo manualmente.

Beneficio: El manejo del token se centraliza en un solo lugar. Si el esquema de autenticación cambia, solo se modifica el interceptor.

2. Arquetipos Maven

2.1 Arquetipo utilizado

Todos los microservicios fueron generados con el arquetipo oficial de Spring Boot:

```
groupId:    org.springframework.boot
artifactId: spring-boot-starter-parent
version:    4.0.5 (SNAPSHOT)
```

Este arquetipo proporciona:

- Estructura estándar `src/main/java / src/test/java`
- Gestión de versiones de dependencias vía BOM (Bill of Materials)
- Plugin `spring-boot-maven-plugin` para generar el JAR ejecutable (fat JAR)
- Configuración de Surefire para ejecutar tests con JUnit 5

2.2 Estructura generada por el arquetipo

```
{microservicio}/
├── src/
│   ├── main/
│   │   ├── java/           → código fuente
│   │   └── resources/
│   │       └── application.yml
│   └── test/
│       └── java/           → tests unitarios
└── pom.xml
```

2.3 Dependencias clave por microservicio

Dependencia	Uso
<code>spring-boot-starter-web</code>	Controladores REST
<code>spring-boot-starter-data-jpa</code>	Repositorios JPA
<code>spring-boot-starter-security</code>	Filtros JWT
<code>spring-cloud-starter-circuitbreaker-resilience4j</code>	Circuit Breaker

Dependencia	Uso
<code>spring-cloud-starter-gateway</code>	Enrutamiento BFF (solo Gateway)
<code>spring-kafka</code>	Eventos asíncronos (Orden → Inventario)

3. Resumen de patrones por capa

Capa	Patrón	Implementación
Backend — Dominio	Factory Method	<code>OrdenFactory</code>
Backend — Persistencia	Repository	<code>JpaRepository</code>
Backend — Resiliencia	Circuit Breaker	Resilience4j en clients
Backend — Seguridad/Enrutamiento	BFF	Spring Cloud Gateway
Frontend — Estado	Observer	<code>BehaviorSubject</code> en <code>OrdenService</code>
Frontend — Seguridad	Interceptor	<code>JwtInterceptor</code>